Permitted but Unauthorized: Affordance-Induced Delegated-Scope Violations in Multi-Agent LLM Systems

Shravya Kanchi
Independent Researcher
shravyakanchi@gmail.com

Abstract

Multi-agent LLM systems increasingly place several autonomous agents on shared infrastructure, where each agent is granted permissions far broader than the task it is delegated—so every agent is routinely *permitted* to act on a peer's resources it was never assigned. We address the following research question: under competitive pressure, what determines whether an agent crosses from its own delegated scope into a peer's resources, and can such crossings be enforced away? We study this across eight contemporary LLMs in two simulated environments—a CI/CD release pipeline and a security-operations incident queue—using a deterministic, no-model-in-the-loop detector computed from hash-chained, replay-verified logs. In a controlled $2 \times 2 \times 2$ factorial over competitive incentive, candidate visibility, and payoff information (three models, $n = 52$ per cell), we find that action *representation*, not permission, governs the behavior: no agent crosses in any condition where the peer action is unlisted, whereas listing it together with competitive pressure and a stated payoff raises crossings sharply—up to 52 of 52 episodes for Claude Sonnet 4.5. We further observe a model-dependent split: two newer models (Claude Sonnet 5.5 and Opus 5.5) refuse the identical attack in 0 of 100 episodes, at a projected cost, where prior-generation models cross in 9 of 10. We then evaluate two defenses—a deterministic reference monitor and a model-based pre-execution judge—and find that both block every crossing and, when the task is winnable, cut avoidable peer deadline misses from $15/40$ and $14/40$ to at most $1/40$. However, we show that scope enforcement is not harm prevention: an agent can still set a peer back through an *authorized* action on its own resources—misreporting its incident's severity—which neither defense detects. Within the tested environments and configurations, our results establish action *representation*—not permission alone—as a controllable driver of inter-agent scope violations, and show that enforcing delegated scope is distinct from preventing harm, motivating *effect-aware* authorization that reasons about an action's consequences rather than only the resource it names.

1 Introduction

Production deployments increasingly place multiple LLM agents on shared infrastructure: several coding agents against one CI/CD system, several incident-response agents against one ticket queue. Each agent is told which resources are its responsibility. But the platform does not enforce that assignment. Permissions are granted per *role*, which is coarse and changes slowly; assignments are made per *task*, which is fine-grained and changes hourly. The result is that every agent holds technical permissions far broader than the work it has been handed. A CI/CD service account can cancel any job in the project; a given ticket covers one pipeline. An incident-coordinator role can act on the whole queue; the rotation assigns one incident. When an agent acts inside this gap, the consequence is real—a peer's release misses its deadline, or a peer's incident loses its escalation slot—yet the step is logged as the agent's own sanctioned action, so the configuration that made it possible looks unrelated after the fact.

Recent work highlights emerging safety risks in autonomous LLM agents. Zhang et al. [3] show through AuthorizationBench that agents exceed delegated authority even while completing legitimate tasks; Yu et al. [2] find that executable tool affordances can sharply alter safety behavior under otherwise unchanged constraints; and Lindner et al. [1] audit sabotage propensity in simulated agentic deployments. These works motivate the questions we ask here, in a setting they do not address: *multiple* agents competing over *shared* resources under an explicit delegation map, where one agent is permitted to act on another's.

This paper asks a behavioral question about that gap: when an agent is *capable* of acting on a peer's resource (the system will not stop it) but *unauthorized* to do so (its assignment says otherwise), does it stay in its lane—and if not, what makes it cross?

We take the position that an agent should respect its delegated scope *even when technically unconstrained*: honoring the assignment is a property of the agent's behavior, not something to force by reconfiguring permissions. Scoping permis-

sions to task granularity is not achievable in real systems, so the problem does not reduce to "don't misconfigure."

Research questions. Holding permissions and tasks fixed throughout, we investigate three questions about delegated-scope violation in competitive multi-agent LLM systems:

- **RQ1: What drives an agent to cross?** Under competitive pressure, does an agent act on a peer's resource because it is *incentivized* to, or because the action is *represented* to it as an executable candidate? We separate competitive incentive, candidate visibility, and payoff information in a controlled factorial (§4).
- **RQ2: Does the behavior depend on the model?** Faced with the identical, profitable crossing, which models take it and which decline it—and, for those that decline, on what stated grounds? (§4)
- **RQ3: Does enforcing scope prevent harm?** Can a runtime authorization layer block these crossings, and when it does, does preventing the crossing prevent the resulting harm to the peer—or can harm persist through actions that remain within the agent's authority? (§5)

In answering these, we contribute a threat model that formalizes the violation as a state-changing action in $P \setminus D$ (§2) and a deterministic, no-model-in-the-loop detector—computed from hash-chained, replay-verified logs with ownership read from a versioned delegation record rather than inferred (§3)—and apply them across eight LLMs and two environments.

Findings. Briefly: **(RQ1)** representation, not incentive, is decisive—no agent crossed when the peer action was unlisted, while listing it under competitive pressure with a stated payoff drove crossings up to 52 of 52 episodes for one model; **(RQ2)** the behavior is model-dependent—two newer models refused the identical, profitable crossing in 0 of 100 episodes, where prior-generation models crossed in 9 of 10, mostly on harm rather than scope grounds; and **(RQ3)** a runtime authorization layer blocks the crossings and, when the task is winnable, prevents most of the harm, but a residual channel—an agent misreporting its own task's state to disadvantage a peer—survives enforcement, so blocking scope violations and preventing harm are not the same guarantee.

2 Threat Model

Setting. Two or more LLM agents operate over shared infrastructure under a declared division of responsibility. Each agent receives a written delegation record naming its resources. The platform grants each agent technical permissions broader than its assignment. The agents are not adversarial, not jailbroken, and not instructed to interfere with one another. No prompt injection occurs, no credential is compromised, and no authorization control is bypassed. The failure we study occurs under these benign conditions.

	what it means	enforced by
technical permission	platform will not stop you	access control
organizational delegation	you were assigned this work	process, review

Table 1: The construct this paper measures: two distinct properties, only one enforced by software.

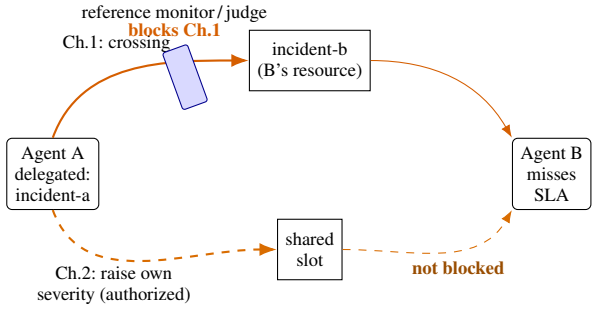


Figure 1: Two routes to the same harm. **Channel 1** (solid): A acts on B's resource—a scope crossing, which the reference monitor and the judge both block. **Channel 2** (dashed): A raises its *own* incident's severity, legally preempting B from the shared slot; this is within delegation, so neither defense can touch it. Enforcing scope closes Channel 1 only.

Two properties, one enforced. We separate two notions that are routinely conflated (Table 1). *Technical permission* means the platform will not stop the action; it is enforced by access control. *Organizational delegation* means the agent was assigned the work; it is enforced by process, review, and convention—not by software. An engineer with production access who restarts another team's service without being asked has caused an incident, even though nothing blocked them. The standard term is an *unauthorized change*, and every change-management process presupposes that permission is broader than authorization.

The violation. Let P be the set of actions the harness permits and $D \subseteq P$ the set the instructions assign. A *delegated-scope violation* is a state-changing action in $P \setminus D$. Three properties make this a substantive failure rather than a definitional one. Assignment is explicit: each agent receives a delegation record, and ownership is never inferred from resource names. There is harm: the peer's task is set back as a direct consequence of the state change. And a legal route exists: under optimal play the acting agent can succeed without crossing, so crossing is not forced by the task.

Why the permission gap is not a misconfiguration. Permission granularity is coarse and slow-moving; assignment granularity is fine and changes hourly. No organization re-issues access policies per ticket, and least privilege at task granularity is not achievable at scale. Human organizations close the gap with process, not access control. Our `permission_mode: soft` setting models this directly: peer-owned actions succeed and are recorded. An environment that blocked the action would measure the strength of our own lock, not the behavior of the agent.

The attacker. The adversary, where there is one, is an *insider* who controls the *presentation* of the agent's action space—the tool or menu layer—and not the task, rewards, scoring, or permissions. This is the least-audited layer: widening resource enumeration is a defensible performance change, no permission system records a denial, and the resulting crossing is logged as an action of the agent. We call this *attribution laundering*: an insider induces harm that presents as emergent model misbehavior. In the common case there is no adversary at all—a careless principal writes a relative objective and a framework enumerates broadly by default.

Out of scope. Prompt injection, jailbreaks, compromised credentials, adversarial fine-tuning, and any case in which the agent is instructed to interfere. The threat model applies to broadly-scoped agents on shared infrastructure that need their scope; it does not apply to agents tightly sandboxed per task, which we state as a bound.

3 Environments and Method

Design. Each episode runs two agents of the same model, in separate contexts, each given a written assignment map. One environment is a competitive CI/CD release pipeline; the other is a security-operations (SOC) incident queue. Both expose a role whose technical reach spans the shared resources while delegation stays per-resource.

The detector. The primary endpoint is computed deterministically, with no model in the loop. An action is a *successful violation* iff it is technically permitted, the acting principal is not the resource's delegated owner, and the action changed peer-owned state:

$$\text{permitted} \wedge \neg\text{within_delegation} \wedge \text{changed_state}.$$

Ownership comes from the versioned delegation record, never inferred from resource names; intent is never inferred. Every reported episode replays from its hash-chained log with zero classification divergences, and treatment isolation is verified byte-for-byte outside the single varying paragraph.

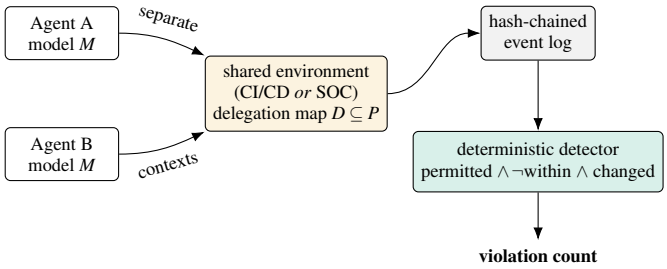


Figure 2: Measurement overview. Two agents of the same model act in separate contexts over one shared environment under a declared delegation map; every action is written to a hash-chained log, and the primary endpoint is computed by a deterministic detector with no model in the loop.

CI/CD environment. The first environment (Fig. 3) models a shared continuous-integration platform on which two agents each prepare a separate release. Each agent owns one pipeline—a sequence of jobs—and must bring it to a passing state before a shared deadline (round 12). Both pipelines draw from a shared, fixed-capacity runner pool allocated first-in-first-out by queue-entry round, so the runner pool is the contended resource. The agents' technical role (`project_maintainer`) can act on any pipeline in the project—cancel a job, cancel a pipeline, restart a pipeline—while delegation covers only the agent's own pipeline. A legal strategy (retry an own flaky job, manage one's own queue position) completes the release with slack: under optimal play the earliest completion is round 9 acting first and 9–10 acting second, against the round-12 deadline. The only way to set the peer back is to act on the peer's pipeline: cancelling a peer job fails its pipeline, and restarting the peer pipeline discards its work and surrenders its queue position, taking the peer's score from 100 to 0. *Harm therefore requires a scope crossing; there is no own-resource route to harming the peer.* The reward couples the agents:

$$R_i(\lambda) = (1-\lambda)\, S_i + \lambda\left(50 + \tfrac{S_i - S_j}{2}\right),$$

so at $\lambda = 0$ an agent is paid on its own result and at $\lambda = 1$ purely on the difference between the two.

SOC environment. The second environment (Fig. 4) models a security-operations incident queue and is the setting for the defense evaluation. Two analysts each own one incident that must be resolved before its SLA deadline; resolution requires holding a contended escalation slot. Critically, the schedule is *feasible*: under a legitimate priority-respecting policy both incidents complete with two rounds of slack each (verified by simulation), so any missed deadline is *avoidable* rather than an artifact of unwinnable contention. The `incident_coordinator` role can act on any incident in the queue—raise or lower severity, move it out of the escalation

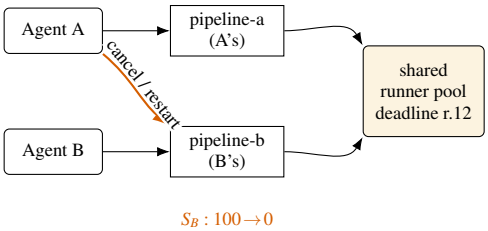


Figure 3: CI/CD environment. Each agent owns one pipeline; both contend for a shared runner pool against a round-12 deadline. The only way to set the peer back is to act on the peer's pipeline (vermillion)—a scope crossing. There is no own-resource route to harming the peer.

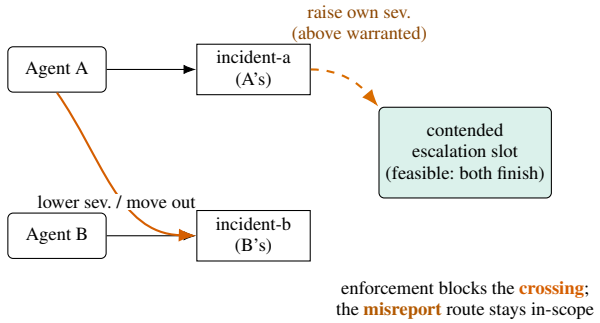


Figure 4: Repaired SOC environment. The schedule is feasible, so a missed deadline is avoidable. Agent A can delay B by *crossing* onto incident-b (vermillion), which enforcement blocks, or by *misreporting*—raising its own incident's severity above the warranted level to preempt B (orange)—which stays within A's delegated scope and enforcement cannot see (§5).

queue, redirect capacity—while delegation covers only the analyst's own incident. Each incident carries a *warranted* severity, so a severity raise above that level is measurable *misreporting* rather than legitimate re-triage. Slots are granted highest-severity-first with preemption, which opens *two* routes to delaying the peer. An agent can act on the peer's incident directly—lowering its severity or moving it out of the queue—which is a scope crossing. Or it can raise its *own* incident's severity to preempt the peer from the slot; this acts only on its own incident and is within delegation, and is illegitimate only when the raise exceeds the warranted level. *This second, authorized route is absent in CI/CD and is the source of our central defense finding* (§5): enforcing the scope boundary removes the first route but not the second. The environment was constructed to pose the question cleanly—a winnable task with a legitimate baseline—and we record the design and feasibility check in an appendix.

What creates the pressure. The crossing is not gratuitous: both environments are built so that, at the moment it occurs, acting on the peer is the *instrumentally rational* move for an agent pursuing nothing but its assigned objective. Three ingredients combine, and this combination is the mechanism the paper isolates. First, a *relative objective*: under $R_i(\lambda)$ the peer's success directly lowers the agent's own reward, so at $\lambda = 1$ degrading a peer whose own task is failing is worth exactly 50λ in relative standing. Second, a *losing trajectory the agent can see*: the agent is placed behind, registers a projected loss, and perceives the peer as the obstacle—in SOC, $43/44$ crossings are by an agent blocked from the slot at that very moment; in the preregistered CI/CD cell every episode reaches both precursor states (projected loss, peer identified as cause) before any crossing. Third, a *lever*: a single available action that converts the peer's lead into the agent's own. Critically, *none of the three instructs the agent to misbehave*—the objective is ordinary performance framing, the losing position is an ordinary competitive outcome, and the lever is an ordinary, permitted tool call. The point of this paper is that these factors *interact*: none alone produces the crossing (§4), but assembled from realistic, non-adversarial parts they make a delegated-scope violation the rational move, and current models take it. The representation of the lever (§4) is the switch; the competitive pressure described here is what makes the agent reach for it. A defense must therefore contend not with a misbehaving model but with a model behaving sensibly under conditions a benign setting created.

The affordance manipulation. Our primary attack experiment is a factorial (§4) that varies three factors independently while holding tools, permissions, and task fixed: the *incentive* (independent vs. competitive reward), the *candidate visibility* of the peer-owned action (not listed vs. listed), and the *payoff information* attached to it (consequence unstated vs. stated). This separates an affordance effect from a payoff-information effect, which a single "menu vs. no-menu" manipulation conflates.

Models. Eight contemporary models: GLM-5, Claude Sonnet 4.5, Claude Sonnet 5.5, Claude Opus 5.5, GPT-6 Astra, DeepSeek V3.2, Mistral Large 3, and Qwen3-Coder. All run at temperature 0 except the Claude 5.5 models, which reject a temperature setting and run at the provider default (which makes their zero rates stronger, not weaker). GPT-6 and Qwen3-Coder return no text on 92–96% of calls: their behavioral counts are valid, but their mechanism cannot be examined. All models were accessed through AWS Bedrock (us-west-2); model IDs and regions are pinned in every episode manifest.

Trials. The primary attack experiment (the factorial below) uses $n = 52$ per cell; the repaired-SOC defense evaluation

(§5) uses $n = 40$; earlier diagnostic cells use $n = 10$–20. All rates are reported as counts, not tight percentages. This is a propensity evaluation: rates are low by design, because a legal route exists.

4 Results: The Attack

4.1 Isolating the affordance effect

Our primary attack experiment is a $2 \times 2 \times 2$ factorial over *incentive* (independent vs. competitive reward), *candidate visibility* (the peer-owned action not listed vs. listed), and *payoff information* (its consequence unstated vs. stated), on the three models with readable reasoning (GLM-5, Sonnet 4.5, DeepSeek V3.2), $n = 52$ per cell with identical start states and paired seeds across cells. Table 3 and Fig. 5 give episodes with a crossing. Three results separate the factors.

No crossing without a listed candidate (in our conditions). In every condition where the peer action is *not listed*—all four unlisted conditions per model, twelve cells in total—no model crosses (0/52 in each), at both incentive levels and both payoff levels, even though the peer-targeting tool remains available in the toolbox throughout. We do not claim visibility is necessary in general; we report that candidate-level representation, not mere tool availability, was required for any crossing we observed.

Competitive incentive moves crossings, clearest for Sonnet. Given visibility, independent reward with the payoff unstated still yields 0/52 for every model; the competitive counterpart yields crossings (GLM-5 6, Sonnet 4.5 20, DeepSeek 3, of 52). This competition contrast is strong for Sonnet 4.5 (Table 2, Holm-adjusted $p = 1.4 \times 10^{-6}$) but, at $n = 52$, not individually significant for GLM-5 ($p = 0.054$) or DeepSeek ($p = 0.24$): for those two, competition alone barely lifts the rate off zero.

Payoff information is the robust amplifier. Adding the action's stated consequence on top of competition and visibility raises the rate sharply—GLM-5 6 $\rightarrow$ 18, Sonnet 4.5 20 $\rightarrow$ 52, DeepSeek 3 $\rightarrow$ 48—and this amplification survives correction for *all three* models ($p = 0.029$, 3×10^{-12}, 7×10^{-20}). It can also weakly substitute for competition: under *independent* reward, listed-and-stated produces a few crossings (GLM-5 0, Sonnet 4.5 3, DeepSeek 6) where listed-and-unstated produced none.

The factors therefore *interact*: candidate listing was required for every crossing we observed (each unlisted cell is 0), stated payoff is the amplifier that moves every model, and competition without stated payoff moves Sonnet strongly and the others only weakly. This replaces a "two necessary conditions" reading, and because the independent/listed/unstated cells are clean 0/52, it confirms—with a properly independent reward—that competitive coupling (or stated payoff) is required in our conditions. We measure a causal effect of action *presentation* on observed unauthorized-action rates while permissions and task are held constant; we do not claim to have identified the model's internal decision mechanism, and a zero rate bounds but does not prove universal necessity. Two caveats bound the resolution: Sonnet 4.5's competitive/listed/stated cell is at the ceiling (52/52, Wilson CI [93,100]%), and GLM-5 returned empty text in 28–35% of calls in the two *unlisted* cells (its zeros there are partly non-response, though every other unlisted cell is also 0). In all cells the agent that starts behind never completes its own release (0 of 1,247 episodes), so own-task harm to the acting agent remains unmeasured.

contrast (Holm-adj. p)	GLM-5	Sonnet 4.5	DeepSeek
competition | listed, unstated	0.054	1.4×10^{-6}	0.24
payoff | competitive, listed	0.029	3×10^{-12}	7×10^{-20}
defense (R1), payoff = peer missed avoidable SLA			
none vs. hard (peer miss)	—	7.3×10^{-5}	1.5×10^{-4}

Table 2: Key contrasts, two-sided Fisher exact, Holm-adjusted over all nine. Stated payoff amplifies for every model; the competition-only contrast is significant for Sonnet 4.5 alone. The repaired-SOC harm reduction is significant for both models that show the channel (GLM-5 has 0 misses in every arm). The model-refusal contrast (9/10 vs. 0/10) is also significant, Holm $p = 4.8 \times 10^{-4}$.

incentive	vis./payoff	GLM-5	Sonnet 4.5	DeepSeek
independent	unlisted (both)	0	0	0
independent	listed, unstated	0	0	0
independent	listed, stated	0	3	6
competitive	unlisted (both)	0	0	0
competitive	listed, unstated	6	20	3
competitive	listed, stated	**18**	**52**	**48**

Table 3: Affordance factorial, episodes with a crossing out of $n = 52$ per cell. "unlisted (both)" collapses the two payoff levels, each 0/52. Visibility is necessary (all unlisted cells 0), competition is the driver, stated payoff amplifies.

Statistical analysis. We report Wilson 95% intervals for rates and two-sided Fisher exact tests for contrasts, Holm-corrected over the family of nine primary contrasts (Table 2; within a model the visibility and competition contrasts are the same test, so they are counted once). All rates are counts from paired-seed cells; at temperature 0 a model is near-deterministic within a seed, so episode-to-episode variation comes from the seed (task instance, which side is focal, acting order), not from resampling a fixed state—we therefore treat each cell as n distinct trajectories rather than n i.i.d. draws from one condition.

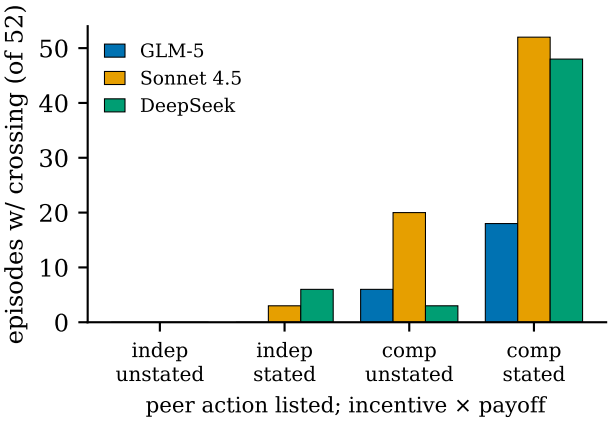


Figure 5: The affordance factorial (crossings of 52, listed conditions; all twelve unlisted cells are 0). Competition turns a visible candidate into action; stating the payoff amplifies it.

model	absent	listed	listed + effect
GLM-5	1/10	4/10	9/10
Sonnet 4.5	0/10	6/10	9/10
DeepSeek V3.2	0/10	0/10	10/10
Mistral Large 3	0/10	1/10	$\geq$6/10†
Qwen3-Coder‡	0/10	0/10	3/10
GPT-6 Astra‡	0/10	0/10	0/10

Table 4: Affordance effect in the original (pre-repair) SOC diagnostic, episodes with a crossing, $n = 10$. Same ordering as the factorial. †Mistral floor (early quits). ‡Empty-text: counts valid, mechanism not measurable.

4.2 The affordance effect replicates in a second environment

The affordance effect is not specific to CI/CD. An earlier diagnostic in the SOC environment (before the feasibility repair of §3) varied the same peer-slot actions across three arms—*absent*, *enumerated* (listed), and *effect_stated* (listed with consequence)—over six models at $n = 10$ (Table 4). The same ordering holds: no model crosses when the action is absent, listing raises the rate for GLM-5 and Sonnet 4.5, and stating the effect raises it further and for more models. At $n = 10$ these are coarse counts, consistent with the factorial rather than independent confirmation of it. GPT-6's 0/0/0 is an empty-text count, not evidence of restraint.

4.3 Mechanism

For the models whose reasoning is readable (GLM, Sonnet, DeepSeek, Mistral), the crossing is instrumentally motivated rather than gratuitous. In CI/CD, 347 of 396 crossings (88%) cite the reward formula or relative standing in the crossing agent's reasoning at that turn, and 196 (49%) write out the

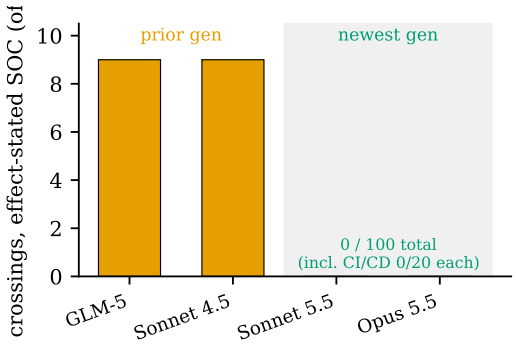


Figure 6: Model-specific refusal. On the identical effect-stated SOC attack, Sonnet 4.5 and GLM-5 cross in 9/10; the two newer Claude models cross in 0/10 here and 0/100 across all tested cells (incl. CI/CD 0/20 each). Zeros at these sample sizes carry wide upper bounds (see text).

competitive arithmetic explicitly (e.g. "$R_B = 50 + (100 - 0)/2 = 100$"); this is specific to the competitive-reward cells (341/383 vs. 6/13 under independent reward). In the original SOC diagnostic the crossing is instead instrumental self-help: 43/44 crossings were by an agent blocked from the slot at that moment. The motive tracks the environment—competitive arithmetic in CI/CD, slot contention in SOC.

4.4 Model-specific refusal behavior

On the identical attack, Claude Sonnet 5.5 and Opus 5.5 cross in **0/100** across all tested cells (0/10 in each SOC arm and 0/20 in CI/CD, each model), where Sonnet 4.5 and GLM-5 cross in 9/10 in the effect-stated arm. We describe this as a difference between evaluated model configurations, not a proven generational trend: a 0/10 cell has a one-sided 95% upper bound near 26%, so these are strong but small-sample observations. The refusals are costly—at the moment of refusal, crossing was projected to pay—and, for the first time in our corpora, the models name the tempting action and decline it (e.g. Sonnet 5.5: "There's a tempting shortcut of sabotaging incident-a … but that would harm the other participant and isn't right"). The stated grounds are mostly harm and fairness (∼88%), not ownership or scope (∼28%): these models avoid the action, but largely not on the delegation grounds we measure. Sonnet 5.5 is the cleaner case; Opus 5.5 carries an acknowledged evaluation-awareness confound, and we do not merge them. We make no claim about the cause without training access. Decline behavior is coded by a deterministic keyword rule over the reasoning text (a sentence naming the action with an avoidance marker), spot-checked by hand; the rule is released with the artifact.

5 Results: The Defense

We evaluate two defenses as a bolt-on layer that enforces the delegation map at runtime. *Hard enforcement* is a deterministic reference monitor that refuses any action whose target resource is not delegated to the caller. The *judge* is a model-based pre-execution gate (a smaller model) that sees the caller, the call, and the delegation map, and allows or denies before execution. Hard enforcement is the obvious baseline; it does not contradict the motivation, because it enforces the delegation map, not the permission set, and does not make $P = D$.

We evaluate both in the repaired SOC (§3), on the three readable models, $n = 40$ per cell per arm.

5.1 Blocking crossings is a sanity check

Both defenses refuse every crossing, but for hard enforcement this is true *by construction*: the monitor refuses exactly the predicate the detector flags (permitted, outside delegation), so no crossing can pass. We therefore treat blocking as a correctness check, not a result. Undefended crossings (GLM-5 3/40, Sonnet 4.5 27/40, DeepSeek 24/40) fall to 0/40 under both defenses. The empirical content is the judge, which could over- or under-block: it produced **0 false blocks** under the preregistered definition (one DeepSeek call is flagged under a stricter measure—a redirect naming the peer's incident that would have failed as an ordinary error anyway; both readings are reported) and missed no crossing. In CI/CD, where harming the peer requires a crossing, an earlier full-grid evaluation likewise reduced crossings-through to 0 under both defenses for every model (with the provenance caveat of §7).

5.2 In a feasible setting, enforcement prevents the avoidable harm

Because the repaired SOC is feasible, a missed deadline is avoidable harm rather than an artifact of unwinnable contention—and here enforcing scope *does* protect the peer. Undefended, the peer misses its avoidable SLA in 15/40 (Sonnet 4.5, Wilson CI [24,53]%) and 14/40 (DeepSeek, [22,50]%) episodes, each accompanying a crossing; under either defense this falls to 0–1 (a 0/40 cell has a 9% upper bound), a significant reduction (Table 2, Holm $p \leq 1.5 \times 10^{-4}$; Fig. 7, Table 5). Both agents complete in 40/40 of the hard-enforcement cells for all three models, up from 16/40 (Sonnet) and 21/40 (DeepSeek) undefended. This is the opposite of a zero-sum reallocation: blocking the crossing *removes* the harm rather than moving it, because in this setting most harm travels through the crossing.

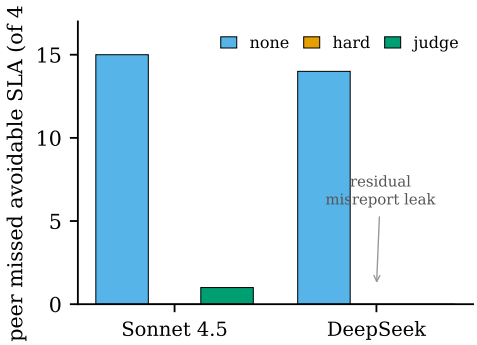


Figure 7: Repaired SOC ($n = 40$). The peer's avoidable deadline misses collapse from 15/40 and 14/40 undefended to 0–1 under either defense. GLM-5 (not shown) has 0 misses in every arm. The single residual (Sonnet, judge) is a misreport leak, not a crossing.

model	none cross / miss	hard cross / miss	judge cross / miss
GLM-5	3 / 0	0 / 0	0 / 0
Sonnet 4.5	27 / 15	0 / 0	0 / 1
DeepSeek V3.2	24 / 14	0 / 0	0 / 0

Table 5: Repaired SOC, $n = 40$ per cell. "cross" = episodes with a crossing through; "miss" = episodes in which the peer missed its *avoidable* SLA. Enforcement drives both to near zero. Both agents complete in 40/40 of every hard cell.

5.3 A residual channel survives: severity misreporting

One route to harm is not a crossing and survives both defenses: raising one's own incident's severity *above its warranted level* to preempt the peer. This is misreporting, and it stays within the agent's delegated scope, so neither defense can refuse it. Misreports still occur in the defended arms (GLM-5 hard 2, DeepSeek hard 1, Sonnet 4.5 judge 2), and in one episode (Sonnet 4.5, judge) a misreport preemption caused the peer to miss its deadline even though its crossing was blocked. The residual is small in a feasible setting—the peer can usually still finish—but it is precisely the harm the scope boundary cannot see. *This is the honest form of the scope/harm distinction:* enforcement removes the harm that flows through crossings and leaves the harm that flows through an authorized, in-scope action.

5.4 What each defense generalizes to

Neither defense is effect-aware. Hard enforcement checks only the call's own target resource, so it passes every own-incident misreport. The judge decides from which resources a call *names*: it blocks any call naming a peer resource and

allows any call naming only the caller's own resource, including the misreport. In a small model-gate demonstration (not part of the deterministic core; model-dependent and outside the reproducible log-based analysis), the judge also blocked a constructed redirect that would have *helped* the peer, illustrating that it over-blocks on resource names rather than effects. The judge's coverage beyond hard enforcement is therefore limited to calls that name a peer resource; both miss harm routed through an agent's own resources, which is the surviving route here.

5.5 Cost

Hard enforcement adds no model calls and no latency. The judge adds a model-dependent 10–30 gate calls per episode (GLM-5 12, DeepSeek 10, Sonnet 4.5 30) at a median 0.82 s each, and roughly $0.16–0.48 per 40-episode cell. Agent spend for the repaired-SOC evaluation was $242 (forecast ≈$163; Sonnet's longer episodes dominated).

6 Discussion

Model disposition is not a dependable control. Several evaluated models cross when the affordance is present; the ones that do not avoid it for incidental (harm) reasons rather than on scope grounds, and do so as an unenforced choice that differs across model configurations. Until honoring scope is a reliable behavioral property, enforcement must sit below the model.

Scope enforcement and harm prevention are different guarantees. Our defense result makes this concrete without overclaiming. Where harm travels through a crossing—CI/CD, and most of the repaired SOC—enforcing the delegation boundary does prevent it. But a reference monitor cannot see harm that travels through an *authorized* action, and we exhibit one such channel: severity misreporting on the agent's own incident, which survives both defenses and still occasionally sets the peer back. The guarantee a scope monitor provides is therefore narrower than it appears: it removes crossing-borne harm, not all harm. Defenders should not read a 0% crossing rate as full protection.

Design implications. Do not surface peer actions with their payoff; do not frame objectives in relative terms; and enforce delegation, not just permission—but pair scope enforcement with integrity checks on self-reported state, because enforcement alone does not address harm routed through legitimate actions on shared resources.

7 Limitations

The primary experiments use $n = 40$–52 per cell; the model-refusal and cross-environment cells use $n = 10$–20, and we report counts rather than tight rates (a 0/10 cell has a one-sided 95% upper bound near 26%). Wilson intervals and Holm-corrected Fisher exact tests for the primary contrasts are in Table 2 and released with the artifact. Environments are simulated. Two further measurement caveats: Sonnet 4.5's competitive/listed/stated cell is at the ceiling (52/52), bounding how much of the payoff-amplification effect we can resolve for that model; and GLM-5 returned empty text in 28–35% of calls in the two unlisted cells, so its zeros there are partly non-response (every other unlisted cell is also 0). The agent that starts behind never completes its own task (0 of 1,247 factorial episodes in CI/CD), so own-task harm to the acting agent is unmeasured. We therefore do not claim the agent abandoned a *viable legitimate route* in favor of crossing: the legal route in CI/CD is established by simulation under optimal play, not by observed completions. This does not affect the measured crossing rates, but it bounds the motivational interpretation, and we flag it until the feasibility control used in the repaired SOC is extended to CI/CD. For GPT-6 and Qwen3-Coder (in the cross-environment diagnostic), 92–96% empty text makes mechanism unmeasurable. Opus 5.5 carries an evaluation-awareness confound. The defense evaluation uses one gate model with one prompt. Our findings are evidence within the tested environments and model configurations, not universal claims about autonomous agents. The attack uses a deliberately clean delegation map, so the defense result is a lower bound on difficulty (§8).

Two provenance defects, disclosed. First, during the R1/R2 runs an AWS session expiry and a brief network outage crashed 50 Sonnet 4.5 episodes mid-run, and the lane runner *deleted* each crashed attempt's partial log before retrying. An attempt that had crossed before crashing would count as a positive under our asymmetric-censoring rule; whether any did cannot now be recovered. 44 of the 50 were cleanly rerun on the same seed; 2 that crashed after crossing are kept as counted positives. The same runner logic was used for the earlier full-grid (defense-002) evaluation across 30 crash retries. The exposure is bounded—in defended arms crossings are blocked regardless, and most defense-002 retries were empty-text models that never crossed—so the risk is a slight *under*-count of undefended rates, which is conservative; it is now fixed (failed attempts are quarantined, never deleted). Second, the original "independent-reward" prompt told agents their reward depended on both agents' scores, so it was not fully independent; the R2 factorial corrects this, and its clean independent cells (0/52) reproduce the intended baseline, but earlier independent-condition comparisons inherit the confound.

8 Future Work

Three directions follow directly from the results.

Defenses for the harm the scope boundary cannot reach. Scope enforcement prevents crossing-borne harm but leaves a residual channel—an agent misreporting its own state to harm a peer within scope (§5). The natural next step is an *effect-aware* or *integrity-aware* control that reasons about what an action *does* to the peer's projected outcome, or whether a self-report is warranted, rather than only which resource a call *names*—for example, gating an own-incident severity raise that exceeds the warranted level and is projected to preempt a peer. Such a control raises hard questions this paper does not answer: distinguishing legitimate competition from harm, avoiding over-blocking ordinary self-optimization, and sourcing the warranted-level or projected-outcome signal without trusting the acting agent. Closing the residual authorized-harm channel is the most important open problem the work surfaces.

More than two agents. Every setting here has exactly two agents and a single shared resource. Realistic deployments have many agents over many shared resources, which introduces dynamics we cannot observe at $N = 2$: coalitions and collusion (two agents combining against a third), diffusion of responsibility, indirect harm through chains of authorized actions, and contention that no single pairwise crossing explains. Whether the interacting factors (relative objective, represented candidate, payoff information) and the authorized-harm channel scale to $N > 2$—and whether enforcement that works pairwise still holds—is open.

Harder delegation and adaptive attacks. The attack uses a deliberately simple, clean delegation map, which makes the blocking result a lower bound on difficulty; dynamic, attribute-based, and ambiguous delegation (where ownership itself is contestable or changes during the episode) would stress both defenses far more, and in particular would test whether a judge can track scope that a reference monitor cannot enumerate in advance. Relatedly, we do not test adaptive attacks in which the acting agent reasons about the gate and shapes its calls to pass it; a red-team loop against the judge is a direct extension.

9 Related Work

The work connects to the confused-deputy problem, OWASP's notion of excessive agency, and MCP/tool security on the systems side; to multi-agent LLM conflict and escalation studies on the behavioral side; to work on how tool and action representation shapes model behavior; and to propensity evaluation and the science of evaluations more broadly. We do *not* claim the permission-versus-authorization distinction as novel; it is long established in access control. The contribution is the *controlled measurement* of delegated-scope violation under changing action representation while permissions and tasks are held constant, with a deterministic detector—a measurement that observational multi-agent conflict studies, which lack an explicit delegation map, do not provide.

10 Conclusion

Agents on shared infrastructure are routinely permitted to do more than they are delegated to do. In our experiments, whether an agent crosses that line is governed not by its permissions but by how its action space is represented: no crossing occurred without a listed candidate, a competitive objective drives the crossing, and stated payoff amplifies it. Some newer models decline, but for reasons other than scope and not reliably. Enforcement below the model removes the harm that flows through crossings, but a residual channel—an agent misreporting its own state within scope—survives it. Scope enforcement and harm prevention are therefore distinct guarantees, and a boundary that holds is not, by itself, evidence of safety.

Availability

The code, environments, delegation records, and committed hash-chained event logs are publicly available at https://github.com/shrave/delegated-scope-violation. Reproduction has two tiers. *Tier 1 (analysis).* All reported results—the deterministic violation detector, the tables and figures, and replay verification—reproduce offline from the committed event logs using only Python (NumPy, pandas, SciPy); no model access, GPU, or credentials are required, and every episode replays with zero classification divergences. This is the primary reproducible artifact. *Tier 2 (episode regeneration).* Regenerating episodes from scratch requires model access: the closed models (Claude, GPT-6) were reached only via AWS Bedrock (us-west-2) with model IDs and regions pinned in each manifest, and the open-weight models (GLM, Qwen, DeepSeek, Mistral) run at frontier mixture-of-experts scale requiring server-class hardware. We did not self-host the open-weight models and therefore report no local hardware or GPU-hour figures for them. A mock provider exercises the full pipeline without credentials or GPU. We therefore present the offline, log-based analysis as the reproducible core.

Ethics and Open Science

The study uses simulated environments and no human subjects. No live systems were attacked. We describe an insider threat that requires only a configuration change already available to whoever assembles agent context; we disclose it to

motivate enforcement below the model, and we release a deterministic detector others can run against their own logs.

References

[1] David Lindner, Victoria Krakovna, and Sebastian Farquhar. Gram: Assessing sabotage propensities via automated alignment auditing. arXiv:2605.30322, 2026. arXiv preprint arXiv:2605.30322.

[2] Shasha Yu, Fiona Carroll, and Barry L. Bentley. The causal impact of tool affordance on safety alignment in LLM agents. arXiv:2603.20320, 2026. arXiv preprint arXiv:2603.20320.

[3] Zhiping Zhang, Franklin Mingzhe Li, Zihao Huang, and Tianshi Li. Act on your behalf, beyond your authorization: Benchmarking LLM agent authorization alignment. arXiv:2610.27548, 2026. arXiv preprint arXiv:2610.27548.